

编译原理课程实践报告

信息科学与技术学院 2100013122 张伟

一、编译器概述

1.1 基本功能

本编译器具备以下基本功能：

将SysY程序翻译为Koopa IR。将SysY程序翻译为RISC-V汇编。提供基本的非法SysY程序识别功能。

二、编译器设计

2.1 主要模块组成

编译器主要模块包括：

main.cpp：负责读取SysY程序并处理用户交互。 sysy.y：定义SysY语言的语法规则。 sysy.l：定义SysY语言的词法规则。 ast.cpp：负责解析AST并生成Koopa IR。 ast.hpp：声明与AST相关的接口。 koopa_visit.cpp：解析内存形式的Koopa IR并生成RISC-V代码。 koopa_visit.hpp：声明与Koopa IR解析相关的接口。

2.2 核心数据结构与模块设计

本编译器的核心功能包括将SysY抽象语法树（AST）转换为中间表示（IR）并进一步生成RISC-V代码。在此过程中，设计了两个主要的类：AST2IRConverter 和 KoopaVisitor，分别用于AST到IR的转换以及IR的遍历与生成。

- AST2IRConverter 类负责将AST中的各个节点转换为IR。其核心数据结构包括：
 - ir_info：用于存储生成的IR信息。
 - symbol_table：符号表，用于管理变量、函数等信息。
 - while_block_stack：用于记录当前循环块的状态。
 - 各类 Convert* 函数用于处理不同类型的AST节点，如ConvertCompUnit、ConvertFuncDef、ConvertAddExp等。

```
class AST2IRConverter {
public:
    std::string Convert(BaseAST *ast); // 主要转换函数
    void init(); // 初始化函数
    IRInfo ir_info; // IR信息存储
    SymbolTable* symbol_table; // 符号表
    std::stack<int> while_block_stack; // 循环块堆栈

    // 各种节点转换函数
    std::string ConvertCompUnit(CompUnitAST *comp_unit);
    std::string ConvertFuncDef(FuncDefAST *func_def);
    std::string ConvertAddExp(AddExpAST *add_exp);
    ...
};
```

- **KoopaVisitor** 类用于遍历IR并生成RISC-V代码。其核心数据结构包括：
 - **risv_code**：用于存储生成的RISC-V汇编代码。
 - **reg_stack** 和 **reg_set**：分别用于管理寄存器栈和已使用的寄存器。
 - **function_stack**：用于维护函数调用栈。
 - 各种 **Visit*** 函数用于处理IR的不同指令类型，如**VisitReturn**、**VisitInteger**、**VisitBinary**等。

```
class KoopaVisitor {
public:
    void Visit(const koopa_raw_program_t &program); // 程序访问函数
    void FunctionPrologue(const koopa_raw_function_t &func); // 函数前言
    void FunctionEpilogue(const koopa_raw_function_t &func); // 函数结尾
    ...

private:
    std::string risv_code; // RISC-V代码
    std::stack<std::string> reg_stack; // 寄存器栈
    KoopaFunctionStack* function_stack; // 函数栈
};
```

2.3 主要设计考虑及算法选择

2.3.1 符号表的设计考虑

在本编译器中，符号表通过SymbolTable类来管理符号信息。每个符号分为常量、变量、函数三种类型，统一用SymbolInfo类型表示，支持常量、变量和函数的信息存储。

为了处理变量作用域，编译器采用了嵌套符号表结构：每个符号表通过parent和child指针连接，表示层次关系。每次查找符号时，会从当前符号表顺着parent一直查找到根节点，直到找到对应符号。对于变量，根据翻译需要细分为函数参数、指针类型、数组类型和简单的整数类型，同时在结构体中保留了对应指针类型的嵌套层级（nested_layer）来完成lv9中相应的翻译(getptr/getelem_ptr)。

```
/* 结构体用于存储常量信息 */
struct ConstantInfo {
    int value;
};

/* 结构体用于存储变量信息 */
struct VariableInfo {
    int nested_layer;
    bool is_param; // 是否为函数参数
    bool is_pointer; // 是否为指针类型
    bool is_array; // 是否为数组类型
    // 其他必要的变量信息
};

/* 结构体用于存储函数信息 */
```

```

struct FunctionInfo {
    std::string type; // 函数的返回类型
};

/* 使用std::variant来表示不同类型的符号信息，可以是常量、变量或函数 */
using SymbolInfo = std::variant<ConstantInfo, VariableInfo, FunctionInfo>;

/* 符号表类，管理和查找符号信息 */
class SymbolTable {
public:
    std::optional<SymbolInfo> getSymbolInfo(const std::string& symbol) const;
    ...

private:
    std::unordered_map<std::string, SymbolInfo> table;
    SymbolTable* parent;
    SymbolTable* child;
};

```

2.3.2 寄存器分配策略

实验采用了最简单的寄存器分配策略：将所有变量存放在栈上。这种方式具有以下几个优点：

1. **Koopa IR 到 Riscv 的翻译更为简化**：由于 Koopa IR 采用 SSA（静态单赋值）形式，计算结果直接存储在栈上并与特定地址相关联。这样就不需要处理寄存器分配和活跃变量的关系，翻译过程容错性更高。
2. **函数调用过程中的参数传递与分配更为便捷**：无需处理复杂的调用者/被调用者寄存器保存问题（尽管自己仍然实现了调用者寄存器的保存和恢复，然后没时间完成更复杂的寄存器分配策略了orz）。

三、编译器实现

3.1 各阶段编码细节

Lv1. main函数和Lv2. 初试目标代码生成

Lv1/Lv2 主要内容是实现main.cpp，支持-koopa/-riscv两种翻译模式；同时根据文档搭建sysy.l/sysy.y的基本框架

耗时：2h-3h

Lv3. 表达式

Lv3 借鉴了语法制导翻译的思想，在每个 ExpAST 类中添加了一个综合属性 ret_val。在前端翻译阶段，根据当前表达式的计算规则，获取子表达式的 ret_val，并生成新的表达式值。

```

/*以MulExpAST为例 */
class MulExpAST : public BaseAST {
public:
    std::unique_ptr<BaseAST> unary_exp;

```

```

std::unique_ptr<BaseAST> mul_exp;
std::string op;
std::string ret_val; /*综合属性 */
bool is_const;

/* 标记推导类别 */
enum class Type {
    UNARY_EXP,
    MUL_EXP
};
Type type;
...

```

同时在 逻辑或 和 逻辑与 的翻译中，由于Koopa IR仅支持位运算，所以最终采用如下翻译

```

result = (operand1 != 0) || (operand2 != 0)
result = (operand1 != 0) && (operand2 != 0)

```

后端没有太多注意的地方，添加VisitBinary函数对koopa_raw_binary_t类型处理表达式即可。

耗时：1天

Lv4. 常量和变量

实际上是前7个Lab中最花时间的一个，主要需要完成两件事：

1. 前端增加符号表来分辨常数和变量两种类型
2. 后端完成所有寄存器在栈上的存储，增加FunctionPrologue和FunctionEpilogue两个函数和从koopa_raw_value_t类型到栈上地址的映射

先说前端，前端增加了SymbolTable 1.0版本，在处理ConstDefAST和VarDefAST时分别向其中插入符号，而符号的读取则主要在LValAST中完成，读取时根据类型完成相应的翻译，同时需要注意常量的求值需要在前端翻译时完成，因此在所有的ExpAST中添加is_const类成员（综合属性），在翻译时若两个operand.is_const均为True则无需增加新的IR，直接求值。

后端在翻译时需要先遍历一遍function的Koopa IR代码，识别出其中需要分配栈空间的地方，添加地址映射，最后计算总体需要的栈空间；在翻译时，需要读取对应栈地址，并进行相应操作。Epilogue同理

```

void KoopaVisitor::FunctionPrologue(const koopa_raw_function_t &func) {
    koopa_raw_slice_t bbs = func->bbs;
    for (size_t i = 0; i < bbs.len; ++i) {
        //identify values to be stored on stack
        ...
    }
    // round up to 16 bytes
    function_stack.stackSizeRoundUp();

    // Note here we decrease the stack pointer
    int stack_size = -function_stack.getStackSize();
}

```

```
...
}
```

耗时：2-3天

Lv5. 语句块和作用域

非常好语法，使语句块和作用域非常清晰（bushi）主要内容就是在`SymbolTable`中添加`parent`和`child`指针，在查找符号时从当前符号表向根符号表查找即可，同时为每一个符号表添加`id`，用于为不同作用域中相同变量添加后缀，避免 Koopa IR 中变量重合

```
std::optional<SymbolInfo> SymbolTable::getSymbolInfo(const std::string& symbol)
const {
    SymbolTable* current = const_cast<SymbolTable*>(this);
    while (current != nullptr) {
        auto it = current->table.find(symbol);
        if (it != current->table.end()) {
            return it->second;
        }
        current = current->getParent();
    }
    return std::nullopt;
}
```

耗时：3-4h

Lv6. if语句

Lv6值得探讨的便是If-Else二义性问题和短路求值如何存储返回值，同时还有一个文档中没有说清晰的坑（后面讨论）

1. 先说If-Else二义性问题，解决办法非常简单——将`IfStmt`和`ElseStmt`拆开就好：

```
Stmt      := IF '(' Exp ')' IfStmt ElseStmt
IfStmt    := Stmt
ElseStmt  := ELSE Stmt
```

2. 短路求值中，由于Koopa IR单赋值的属性，不同分支获得的结果需要存在一个统一的变量中，这里做法是在分支跳转前用`alloc`命令分配一个变量`@x`，在短路求值失败时跳转到相应分支将结果`store`在`@x`中，并在短路求值`end`分支`load @x`获取结果

```
// short-circuit evaluation for LAndExp
ret += "\t" + id_0 + " = alloc i32\n";
ret += "\tstore 0, " + id_0 + '\n';
ret += "\tbr " + operand_1 + ", " + if_block_name + ", " + end_block_name +
```

```
'\n';
ret += if_block_name + ":\n";
// translate if_block
ret += end_block_name + ":\n";
ret += "\t" + id_2 + " = load " + id_0 + '\n';
```

3. 关于文档中没有说清楚的点，则是函数的每一个分支中虽然需要以`jump/br/ret`指令结尾，但是存在分支没有`ret`指令的情况（比如`main`函数没有`return`代码），这时需要根据当前函数的类型手动添加`ret 0(int返回类型函数)/ret (void返回类型函数)`。同时分支遇到了这些命令之后应当停止翻译（`Lv7 break/continue`同理）。

耗时：5-6h

Lv7. while语句

Lv6的简单复制，唯一的区别就是在设置`while_entry/while_body/while_end`标签名时，不要使用当前嵌套层数作为后缀即可（作者这里使用了`while block`的数量作为后缀）

耗时：3-4h

Lv8. 函数和全局变量

让人开始头疼的Lab来了，Lv8的主要任务是完成函数声明、函数栈地址计算和调用前的参数传递

1. 函数声明：在`SymbolTable`中添加`Function`类型的符号即可
2. 函数栈地址计算：这部分实际上文档也没有说清楚。在后端翻译过程中，栈地址的处理可以分为两部分，一部分就是在`FunctionPrologue`中计算当前函数为局部变量分配的地址，这部分是能提前处理好的，而另一部分在调用部分`call`命令这里，为了参数传递，实际上需要单独分配一块空间存储参数和调用者保存寄存器，在调用结束后需要回复栈地址指针。之所以这样做是因为函数调用时需要保证栈地址指针指向的部分紧挨着函数参数，从而完成参数传递。即使提前算好调用所需栈空间，也需要在调用前改变栈地址指针。

```
void KoopaVisitor::VisitCall(const koopa_raw_call_t &call) {
    // we should first modify sp value first
    ...
    int stack_size = ...
    risv_code += "\tadd sp, sp, " + reg + "\n";
    ...
    // save all the caller saved reg in reg_stack
    for (size_t i = 0; i < reg_stack.size(); ++i) {
        ...
        risv_code += "\tsw " + reg + ", " + addr + "\n";
    }
    // save the arguments
    for (size_t i = 0; i < call.args.len; ++i) {
        ...
    }
    // call the function
    ...
    risv_code += "\tcall " + func_name + "\n";
```

```

// ret_value
...
// load all the caller reg in reg_stack
for (size_t i = 0; i < reg_vec.size(); ++i) {
    ...
}
// restore sp value
...
}

```

3. 函数的参数传递：为了避免传递参数的寄存器和地址被修改，在进入函数之后后端首先通过`alloc`和`store`将参数保存在栈上，再进行后续翻译操作

耗时：4天

Lv9. 数组

最后一个Lab，谢谢编译原理在期末季+大四申请季给我整出这么一个Lab来，听我说谢谢你 😊

Lv9的主要任务是完成数组和指针两种类型的支持，我们分为前端和后端分别展开：

1. 前端：Koopman IR的翻译主要引入了类型判断，数组初始化补齐 和 `getelem_ptr/get_ptr`的使用。
 - 1.1 类型判断：在翻译`alloc`时需要根据语法分析树确定当前分配类型，这部分递归遍历即可。
 - 1.2 数组初始化补齐：在遍历初始化数组时，首先检查当前是否为整数，若不是整数，则需要根据当前`index`来确定下一层`layer`的`size`，然后递归遍历，具体细节详见代码
 - 1.3 `getelem_ptr/get_ptr`的使用：在前端翻译时，一直苦于如何辨别使用这两个命令。在仔细观察`sysy`语言规范后发现：`get_ptr`只有在当前为指针类型时用来获得具体指向类型即可。目前的`sysy`程序暂未包括形如`*[*i32, 2]`这样的类型，因此简单的判断变量是否为指针类型即可。

```

std::string AST2IRConverter::ConvertLVal(LValAST *lval_exp) {
    if (lval_exp->type == LValAST::Type::IDENT) {
        ...
        // 处理数组类型的变量
        if (is_array) {
            ret = "\t" + lval_exp->ret_val + " = getelem_ptr @" + ident + postfix + ", "
+ lval_exp->lval_ret_val + '\n';
        }
        // 处理指针类型的变量
        else if (is_ptr) {
            ...
            ret += "\t" + ret_id + " = get_ptr " + lval_exp->ret_val + ", " + lval_exp-
>lval_ret_val + '\n';
            ...
        }
        // 处理普通变量
        else {
            ret = "\t" + lval_exp->ret_val + " = load @" + ident + postfix + '\n';
        }
    }
    // 如果符号是常量，我们需要对数组类型的常量做和变量一致的处理

```

```

else if (std::holds_alternative<ConstantInfo>(symbol_info)) {
    if (is_array) {
        ...
        ret = "\t" + lval_exp->ret_val + " = getelem_ptr @" + ident + postfix + ", "
+ lval_exp->lval_ret_val + '\n';
    }
    else {
        lval_exp->is_const = true;
        lval_exp->ret_val = std::to_string(std::get<ConstantInfo>
(symbol_info).value);
    }
}
}
else {
    ret = "does not have this symbol: " + ident;
}
}
else if (lval_exp->type == LValAST::Type::ARRAY) {
    // 递归处理
}
else {
    return "error lval_exp type";
}
return ret;
}

```

2. 后端：文档另一个没说清楚的地方是`getelem_ptr/get_ptr`应该如何翻译。关于偏移量的计算其实很好理解，但是计算获得的地址究竟应该如何保存？还是保存地址对应的值？经过考虑，我选择在局部变量地址中一律保存计算出来的地址的值，同时`load`命令中判断当前地址是否存储着指向整数的指针，若是则额外从指针指向的地址中获得整数，以达到`load`的目标

耗时：4天

3.2 测试情况说明

我选择使用文档中推荐的额外测试集进行测试。经验证，这些测试集足够涵盖线上各个测试点的情况。

四、实习总结

4.1 收获和体会

通过这次实习，我不仅提升了自己的编程能力，还对实际工程中的项目流程有了更深入的理解。在实习过程中，我参与了多个模块的开发和调试，特别是在处理符号表、编译器优化和代码生成等方面积累了宝贵的经验。我学会了如何将理论知识应用于实际问题中，理解了程序的执行过程和优化策略的重要性。

4.2 学习过程中的难点，以及对实习过程和建议

在实习过程中，最具挑战性的是如何理解和设计栈地址空间，处理起来需要细致的设计和实现，尤其是`Lv8`跨函数调用和`Lv9`数组地址翻译问题。我花了不少时间在这方面，虽然最终克服了困难，但如果能在实习开始时就得到更多关于运行时环境的深入讲解，可能会节省不少时间。

此外，在编译器的代码生成部分，我遇到了一些关于寄存器分配和内存管理的难题。在实际调试时，理解如何将中间表示的指令转换成目标机器指令并确保其高效执行，是一个非常复杂的过程。如果能在实习中提供更多的实例演示，或者更系统的讲解相关原理，会对我的学习帮助更大。